Automatic Clipping: Differentially Private Deep Learning Made Easier and Stronger

Zhiqi Bu, Yu-Xiang Wang, Sheng Zha, George Karypis

zhiqibu@amazon.com; yuxiangw@cs.ucsb.edu; zhasheng@amazon.com; gkarypis@amazon.com

Abstract

Per-example gradient clipping is a key algorithmic step that enables practical differential private (DP) training for deep learning models. The choice of clipping threshold R is shown to be vital for achieving high accuracy under DP but difficult to tune (must be tuned jointly with learning rate).

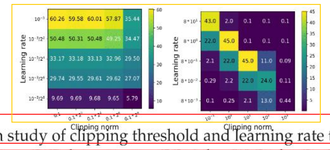

Figure 1: Ablation study of clipping threshold and learning rate that achieves SOTA results. Left: GPT2 on E2E dataset using DP-AdamW. Right: ResNet18 on ImageNet dataset using DP-SGD with momentum.

We propose an easy-to-use replacement, called **Automatic Clipping**, that eliminates the need to tune R for any DP optimizers, while being as private and computationally efficient as existing DP optimizers.

Contributions

- We propose a new per-sample gradient clipping function does not require the clipping threshold R, making hyperparameter tuning of DP deep learning as easy as the regular non-private one.
- We give a convergence analysis of automatic DP-SGD in the non-convex setting, which shows that it enjoys an asymptotic convergence rate $O(T^{-1/4})$ that matches the standard non-private SGD.
- We demonstrate on extensive language (SST-2, QNLI, MNLI, QQP, E2E) and vision tasks (MNIST, FashionMNIST, CIFAR10, ImageNet, CelebA) that automatic clipping outperforms or matches the state-of-the-art accuracy.
- We demonstrate minimal changes to adopt our automatic clipping in existing codebases, e.g. Opacus, ObJAX, Private Transformers, etc.

Differential privacy in deep learning

Differential privacy (DP; Dwork et al.) is a formal definition of privacy that has been shown to prevent the privacy risks in deep learning.

Definition 1. *A randomized algorithm M is (ε, δ)-DP if for any neighboring datasets S, S' that differ by one sample, and for any event E:*

$$\mathbb{P}[M(S) \in E] \leqslant e^{\varepsilon}\mathbb{P}[M(S') \in E] + \delta. \tag{0.1}$$

The key difference between the DP deep learning and the regular one is whether the gradient is privately released. In other words, while the standard optimizers update on the summed gradient $\sum_i \boldsymbol{g}_i$, and DP optimizers update on the *private gradient*:

$$\text{DP Optimizer}(\{\boldsymbol{g}_i\}_{i=1}^B) = \text{Optimizer}(\overbrace{\sum_i \boldsymbol{g}_i \cdot \texttt{Clip}(\|\boldsymbol{g}_i\|; R) + \sigma R \cdot \mathcal{N}(0, \boldsymbol{I})}^{\text{private gradient}}) \tag{0.2}$$

$$\text{Standard Optimizer}(\{\boldsymbol{g}_i\}_{i=1}^B) = \text{Optimizer}(\sum_i \boldsymbol{g}_i) \tag{0.3}$$

Here $\boldsymbol{g}_i \in \mathbb{R}^d$ is the per-sample gradient of loss l_i, $\mathcal{N}$ is the standard normal random variable, σ is the noise multiplier, and R is the clipping threshold. The clipping function $\texttt{Clip}: \mathbb{R}^d \to \mathbb{R}$ is defined such that $\|\boldsymbol{g}_i \cdot \texttt{Clip}(\boldsymbol{g}_i; R)\| \leq R$.

We will show $\boxed{\text{DP Optimizer}_{\text{Abadi}} \approx R\text{-dependent DP Optimizer}_{\text{AUTO}} \equiv R\text{-independent DP Optimizer}_{\text{AUTO}}}$

Per-sample gradient clipping

Abadi's clipping is the most widely applied clipping function, which requires a hyperparameter – the clipping threshold R,

$$\texttt{Clip}_{\text{Abadi}}(\boldsymbol{g}_i; R) = \min(R/\|\boldsymbol{g}_i\|, 1)$$

Recent advances have observed small clipping threshold works best, which motivates our automatic clipping:

$$\texttt{Clip}_{\text{AUTO-V}}(\boldsymbol{g}_i; R) := R/\|\boldsymbol{g}_i\|.$$

We notice that AUTO-V loses the magnitude information of per-sample gradients. We additionally propose AUTO-S with a positive stability constant γ that is set to 0.01 across tasks.

$$\texttt{Clip}_{\text{AUTO-S}}(\boldsymbol{g}_i; R) := R/(\|\boldsymbol{g}_i\| + \gamma).$$

Finally, we can always set $R = 1$, which leads to the actual automatic clipping, which is **R-independent**.

$$\texttt{Clip}_{\text{AUTO-S}}(\boldsymbol{g}_i) := 1/(\|\boldsymbol{g}_i\| + \gamma).$$

Notice that all clipping functions have the same noise-to-sensitivity ratio, hence they have **the same DP guarantee**.

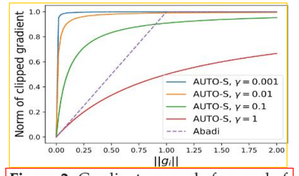

Figure 2: Gradient norms before and after different clipping at $R = 1$.

R-independence of automatic clipping

We show that using automatic clipping, it suffices to tune learning rate and weight decay in DP training as in the regular training.

Theorem 1. *Non-adaptive R-dependent automatic DP-SGD (possibly using momentum) with learning rate η and weight decay λ, is equivalent to R-independent automatic DP optimizers, with learning rate $\eta' = \eta R$ and weight decay $\lambda' = \lambda/R$.*

Theorem 2. *Adaptive R-dependent automatic DP optimizers (including AdaGrad, AdaDelta, AdaMax/Adam, NAdam, RAdam, LARS, LAMB), with learning rate η and weight decay λ, is equivalent to R-independent automatic DP optimizers with learning rate η and weight decay $\lambda' = \lambda/R$.*

Remark 2. *With decoupled weight decay, R-dependent automatic DP-AdamW is equivalent to R-independent automatic DP-AdamW with the same η and λ.*

To prove this, the idea is to couple R with learning rate:

$$R\text{-dep DP-SGD}_{\text{AUTO-S}}: \quad \boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta\Big(\sum_{i \in B_t} \boldsymbol{g}_i \cdot \frac{R}{\|\boldsymbol{g}_i\| + 0.01} + \sigma R \cdot \mathcal{N}(0, \boldsymbol{I})\Big)$$

$$R\text{-indep DP-SGD}_{\text{AUTO-S}}: \quad \boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta'\Big(\sum_{i \in B_t} \frac{\boldsymbol{g}_i}{\|\boldsymbol{g}_i\| + 0.01} + \sigma \cdot \mathcal{N}(0, \boldsymbol{I})\Big)$$

Convergence analysis of automatic DP-SGD

We analyze the convergence on non-convex loss with standard assumptions in SGD literature, and assuming the symmetry of per-sample gradients.

Assumption 3 (Lower bound of loss). *For all $\boldsymbol{w}$ and some constant $\mathcal{L}_*$, we have $\mathcal{L}(\boldsymbol{w}) \geq \mathcal{L}_*$.*

Assumption 4 (Smoothness). *Let $\boldsymbol{g}(\boldsymbol{w})$ denote the gradient of the objective $\mathcal{L}(\boldsymbol{w})$. Then $\forall \boldsymbol{w}, \boldsymbol{v}$, there is an non-negative constant L such that $\mathcal{L}(\boldsymbol{v}) - [\mathcal{L}(\boldsymbol{w}) + \boldsymbol{g}(\boldsymbol{w})^\top(\boldsymbol{v} - \boldsymbol{w})] \leq \frac{L}{2}\|\boldsymbol{w} - \boldsymbol{v}\|^2$.*

Assumption 5 (Gradient noise). *The per-sample gradient noise $\tilde{\boldsymbol{g}}_{t,i} - \boldsymbol{g}_t$ is i.i.d. from some ditribution such that $\mathbb{E}(\tilde{\boldsymbol{g}}_{t,i} - \boldsymbol{g}_t) = 0, \mathbb{E}\|\tilde{\boldsymbol{g}}_{t,i} - \boldsymbol{g}_t\|^2 \leq \xi^2$, and $\tilde{\boldsymbol{g}}_{t,i}$ is centrally symmetric about $\boldsymbol{g}_t$ in distribution: $\tilde{\boldsymbol{g}}_{t,i} - \boldsymbol{g}_t \overset{D}{=} \boldsymbol{g}_t - \tilde{\boldsymbol{g}}_{t,i}$.*

Theorem 3. *Under Assumption 3- 5, running automatic DP-SGD for T iterations with learning rate $\eta \propto 1/\sqrt{T}$ give*

$$\min_{0 \leq t \leq T} \mathbb{E}(\|\boldsymbol{g}_t\|) \leq \mathcal{G}\left(\frac{4}{\sqrt{T}}\sqrt{(\mathcal{L}_0 - \mathcal{L}_*)L\left(1 + \frac{\sigma^2 d}{B^2}\right)}; \xi, \gamma\right) := \min_{r > 0} \frac{\xi}{r} + \mathcal{F}(\cdots; r, \xi, \gamma). \tag{0.4}$$

Here $\cdots$ is the argument of $\mathcal{G}$, which is increasing and positive. As $T \to \infty$, we get $\min_t \mathbb{E}(\|\boldsymbol{g}_t\|) = O(T^{-1/4})$ for AUTO-S, the same rate as standard SGD.

We further take the privacy into consideration to understand how hyperparameters affect the convergence.

Theorem 4. *Under Assumption 3-5 and fixed privacy budget $\mu(\epsilon, \delta)$, running automatic DP-SGD for T iterations with learning rate $\eta \propto 1/\sqrt{T}$ give*

$$\min_{0 \leq t \leq T} \mathbb{E}(\|\boldsymbol{g}_t\|) \leq \mathcal{G}\left(4\sqrt{(\mathcal{L}_0 - \mathcal{L}_*)L\left(\frac{1}{T} + \frac{d}{\mu^2 n^2} + O\left(\frac{1}{B^2 T}\right)\right)}; \xi, \gamma\right) \tag{0.5}$$

Leveraging Theorem 4, our analysis provides guidelines to DP training that match the empirical observation: we minimize the first argument of $\mathcal{G}$ in (0.5), denoted as $X(B, T, \mu, d, L, \mathcal{L}_0)$.

1. **[Train longer with larger noise]** Fixing the expected batch size B, X is decreasing in T. Hence larger T and consequently larger σ are preferred.
2. **[Larger batch size helps]** Fixing number of iterations T or epochs, X is decreasing in B. Hence larger B and consequently larger σ are preferred.
3. **[Pretraining is critical]** Pretraining can leads to a much smaller initial loss $\mathcal{L}_0$ and from a smooth (small L) and flat (small ξ) initialization.
4. **[Learning rate needs tuning]** One should use larger learning rate for smaller model, weaker privacy (larger ϵ or small σ), or larger batch size.

Automatic clipping is easy to code

Switching from Abadi's clipping to automatic clipping changes **only one line of code**. For example in Pytorch Opacus library (v1.1.2), we change line 399-401 in https://github.com/pytorch/opacus/blob/main/opacus/optimizers/ optimizer.py to

```
per_sample_clip_factor = self.max_grad_norm /(per_sample_norms + 0.01)
```

Empirical results

Automatic DP optimizers outperform or match state-of-the-art accuracy, under the same DP guarantee yet tunes no R. Notably, automatic clipping applies compatibly with general optimizers, clipping styles, architecture modifications (e.g. LoRA, RGP, prefix), and data augmentation (e.g. adversarial training).

Task	Model	(ϵ, δ)	Accuracy %		
			Abadi's clipping	AUTO-S clipping	non-DP ($\epsilon = \infty$)
MNIST	4-layer CNN	(3, 1e-5)	98.04 ± 0.09	98.15 ± 0.07	99.11 ± 0.07
FashionMNIST	4-layer CNN	(3, 1e-5)	86.04 ± 0.26	86.36 ± 0.18	89.57 ± 0.13
CIFAR10 pretrained	SimCLRv2	(2, 1e-5)	92.44 ± 0.13	92.70 ± 0.02	94.42 ± 0.01
ImageNette	ResNet9	(8, 1e-4)	60.29 ± 0.53	60.71 ± 0.48	71.11 ± 0.37
CelebA [Smiling]	ResNet9	(8, 5e-6)	90.75 ± 0.11	91.08 ± 0.08	92.61 ± 0.20
CelebA [Male]	ResNet9	(8, 5e-6)	95.54 ± 0.14	95.70 ± 0.07	97.90 ± 0.04
CelebA Multi-label	ResNet9	(3, 5e-6)	86.81 ± 0.03	87.05 ± 0.01	90.30 ± 0.02
CelebA Multi-label	ResNet9	(8, 5e-6)	87.52 ± 0.15	87.58 ± 0.04	90.30 ± 0.02

Table 1: Average test accuracy and 95% confidence interval over 5 runs.

Method	$\epsilon = 3$				$\epsilon = 8$				$\epsilon = \infty$ (non-DP)			
	MNLI	QQP	QNLI	SST2	MNLI	QQP	QNLI	SST2	MNLI	QQP	QNLI	SST2
RGP	-	-	-	-	86.1/86.0	86.7	90.0	93.0	-	-	-	-
full Abadi	**86.43**/86.46	86.43	90.76	93.04	87.02/**87.26**	87.47	91.10	93.81				
full AUTO-V	85.33/85.61	**86.61**	89.99	**93.12**	85.91/86.10	86.86	90.55	93.35	90.33/90.03	87.90	93.61	96.21
full AUTO-S	86.27/**86.67**	**86.76**	**91.01**	**93.92**	**87.07**/87.16	**87.47**	**91.45**	**94.61**				

Table 2: Test accuracy on language tasks with RoBERTa-large (407M parameters).

Metric	DP guarantee	GPT2 large	GPT2 medium	GPT2						
		full AUTO-S	full AUTO-S	full AUTO-S	full AUTO-V	full Abadi	LoRA	RGP	prefix	top2	retrain
BLEU	$\epsilon = 3$	**64.180**	**63.850**	**61.340**	**61.519**	**61.519**	58.153	58.482	47.772	25.920	15.457
	$\epsilon = 8$	**64.640**	**64.220**	**63.600**	63.189	63.189	**63.389**	58.455	49.263	26.885	24.247
	non-DP	66.840	68.500	69.463	69.463	69.463	69.682	68.328	68.845	65.752	65.731
ROGUE-L	$\epsilon = 3$	**67.857**	**67.071**	**65.872**	65.670	65.670	**65.773**	65.560	58.964	44.536	35.240
	$\epsilon = 8$	**68.968**	**67.533**	**67.073**	66.429	66.429	**67.525**	65.030	60.730	46.421	39.951
	non-DP	70.384	71.458	71.359	71.359	71.359	71.709	68.844	70.805	68.704	68.751
NIST	$\epsilon = 3$	**7.937**	**7.106**	**7.071**	**6.697**	**6.697**	5.463	5.775	5.249	1.510	0.376
	$\epsilon = 8$	**8.301**	**8.172**	**7.714**	7.444	7.444	**7.449**	6.276	5.525	1.547	1.01
	non-DP	8.730	8.628	8.780	8.780	8.780	8.822	8.722	8.722	8.418	8.286
METEOR	$\epsilon = 3$	**0.403**	**0.387**	**0.387**	**0.384**	**0.384**	0.370	0.331	0.363	0.197	0.113
	$\epsilon = 8$	**0.420**	**0.418**	**0.404**	0.400	0.400	**0.407**	0.349	0.364	0.207	0.145
	non-DP	0.460	0.449	0.461	0.461	0.461	0.463	0.456	0.445	0.443	0.429
CIDEr	$\epsilon = 3$	**2.008**	**1.754**	**1.801**	**1.761**	**1.761**	1.581	1.300	1.507	0.452	0.116
	$\epsilon = 8$	**2.163**	**2.081**	**1.938**	1.919	1.919	**1.948**	1.496	1.569	0.499	0.281
	non-DP	2.356	2.137	2.422	2.422	2.422	2.491	2.418	2.345	2.180	2.004

Table 3: Test performance on E2E dataset with GPT2.